

HƯỚNG DẪN TOÀN DIỆN VỀ TIẾN TRÌNH NGHIÊN CỨU

Từ Các Thử Nghiệm Thất Bại Đền Khung Tầng Thác Xác Thực (VGC)

Bản Giải Giải Chi Tiết Cho Người Mới Tiếp Cận (Dumb Easy with Deep Notes)

Dự án NCKH: Multi-Step Structured Reasoning · Mô hình: Qwen3-8B-4bit MLX · Thời điểm: 08/10/2026

Abstract

Tài liệu này được biên soạn đặc biệt để bất kỳ ai có một ngày đọc tài liệu đều có thể nắm bắt cặn kẽ toàn bộ câu chuyện khoa học của đề tài: **Tại sao chúng ta làm đề tài này? Chúng ta đã thử những gì và tại sao từng ý tưởng tưởng như “rất hay” lại thất bại thảm hại trong thực tế?** Tài liệu mở xẻ từ gốc rễ: vì sao logprob vô dụng, vì sao cơ chế đồng thuận (voting/agreement) là một cái bẫy tốn kém, tại sao verifier thông minh lại vô tình làm hại hệ thống (*HARM paradox*), và vì sao việc một đoạn code Python thuần túy giải được 100% bài toán lại đặt ra thách thức sống còn cho bài báo khoa học. Cuối cùng, tài liệu làm rõ đặc tả kỹ thuật Contract v3 offline giúp giải quyết triệt để các lỗi hổng trước khi chạy trên tập dữ liệu kiểm chứng độc lập.

Contents

1	Bức Tranh Toàn Cảnh: Đề Tài Này Thực Sự Đang Làm Gì?	2
1.1	Bài toán thực tế trong đời sống của Mô Hình Ngôn Ngữ Lớn (LLM)	2
1.2	Các Chiến Lược Suy Luận Được Đưa Lên Bàn Cân	2
2	Diễn Biến 5 Giai Đoạn Phát Triển: Chúng Ta Đã Thử Gì Và Học Được Gì?	3
2.1	Giai đoạn 0: Chuẩn Hóa Hạ Tầng & Diệt Trừ “Nhiễm Bẩn” (Cold-Starts)	3
2.2	Giai đoạn 1: Ý Tưởng Logprob (Prompt X & Y) — Kết Quả Âm Đắt Giá	3
2.3	Giai đoạn 2: Ý Tưởng Đồng Thuận Dị Thể (Prompt AJ) — Cú Lừa “Verifier Miễn Phí”	3
2.4	Giai đoạn 3: Chạy Thực Tế Trên GPU (Pilot N=2 & N=8) & Phát Hiện Hiện Tượng HARM	4
2.5	Giai đoạn 4: Shadow Replay (Thử Nghiệm Giả Định “What-If”)	4
2.6	Giai đoạn 5: Thách Thức Sống Còn Từ Bộ Giải Hình Thức (Symbolic Solver)	5
3	Bảng Số Liệu Toàn Diện & Hướng Dẫn Đọc Dễ Hiểu	6
3.1	Bảng 1: Kiểm Định Cổng Đón Nhận Đồng Thuận (Agreement Gate)	6
3.2	Bảng 2: So Sánh Hiệu Quả Các Chính Sách (Replay Policies)	6
3.3	Bảng 3: Kết Quả Thử Nghiệm Mô Hình Thật Trên GPU (Pilot N=8)	7
4	Kế Hoạch Triển Khai Contract v3 Offline: Sửa Chữa Tận Gốc	7
5	Tổng Kết 5 Bài Học Đắt Giá Cho Đề Tài NCKH	8

1. Bức Tranh Toàn Cảnh: Đề Tài Này Thực Sự Đang Làm Gì?

1.1. Bài toán thực tế trong đời sống của Mô Hình Ngôn Ngữ Lớn (LLM)

Hãy tưởng tượng bạn thuê một thực tập sinh AI rất thông minh nhưng tính khí thất thường. Khi bạn đưa cho nó một bài toán suy luận phức tạp (ví dụ: giải toán đố nhiều bước, lập lịch thi đấu, hoặc suy luận thứ tự về đích của các vận động viên):

- Nếu bảo nó: “*Nghĩ nhanh rồi trả lời luôn*” (Zero-shot / Chain-of-Thought ngắn): Nó trả lời rất nhanh, tốn rất ít tiền (ít token), nhưng tỉ lệ sai rất cao vì nó nói hớ.
- Nếu bảo nó: “*Hãy suy nghĩ thật kỹ, vẽ cây phương án, thử nhiều trường hợp*” (Tree-of-Thoughts — ToT, hoặc Self-Consistency — SC): Nó trả lời chính xác hơn, nhưng **cực kỳ đắt đỏ và chậm chạp** (tốn gấp 10 đến 20 lần lượng token).
- Nếu bạn lúc nào cũng dùng phương pháp đắt tiền: Ngân sách của bạn sẽ cạn kiệt nhanh chóng.

Trực Quan Hóa: Bài Toán Kinh Tế Của Suy Luận (The Reasoning Economy)

Mục tiêu của đề tài là xây dựng một **Hệ Thống Tầng Thác Thông Minh (Cascade System)**:

- Ban đầu, luôn luôn cho phương pháp **rẻ tiền và nhanh nhất** giải trước (ví dụ: cho LLM viết một đoạn mã Python ngắn — gọi là **PAL**).
- Ở giữa, đặt một **Người Gác Cổng (Gate / Verifier)** để kiểm tra: Lời giải này có đáng tin không?
- Nếu người gác cổng bảo “*Được rồi, lời giải chuẩn xác!*” → Xuất đáp án ngay lập tức (tiết kiệm 80% chi phí).
- Chỉ khi người gác cổng phát hiện “*Chỗ này sai logic, có vấn đề!*” → Mới kích hoạt hệ thống đắt tiền (ToT) để cứu vãn.

1.2. Các Chiến Lược Suy Luận Được Đưa Lên Bàn Cân

Để hiểu các bảng số liệu, bạn chỉ cần nhớ 4 phương pháp chính sau:

- PAL (Program-Aided Language)**: Thay vì bắt LLM tự nhẩm toán bằng lời văn, ta bảo LLM: “*Hãy viết một đoạn script Python ngắn để tính toán/duyet hoán vị*”. Sau đó chạy mã Python đó trên máy. Vì máy tính không bao giờ tính sai số học, PAL chạy cực nhanh và chuẩn xác nếu code đúng.
- CoT (Chain-of-Thought)**: LLM tự suy nghĩ từng bước bằng lời văn tiếng Anh thông thường rồi đưa ra kết luận.
- ToT (Tree-of-Thoughts) / Candidate Selection**: LLM sinh ra 3 nhánh suy luận độc lập (3 ứng viên), sau đó một bộ chọn (Selector) sẽ đọc cả 3 nhánh và chọn ra nhánh tốt nhất. Phương pháp này rất mạnh nhưng tiêu tốn rất nhiều token (gấp 10–20 lần PAL).
- VGC (Verified Generation Cascade)**: Hệ thống phối hợp do nhóm đề xuất: Chạy PAL → Đưa qua bộ kiểm chứng hình thức độc lập (Verifier) → Nếu chứng chỉ hợp lệ thì dừng, nếu không thì leo thang sang ToT.

2. Diễn Biến 5 Giai Đoạn Phát Triển: Chúng Ta Đã Thử Gì Và Học Được Gì?

Dưới đây là hành trình thực nghiệm được ghi chép trung thực theo chuẩn mực nghiên cứu khoa học:

2.1. Giai đoạn 0: Chuẩn Hóa Hạ Tầng & Diệt Trừ “Nhiễm Bẩn” (Cold-Starts)

- Phát hiện bất ngờ:** Khi kiểm tra lại hệ thống đo benchmark trước đây, nhóm phát hiện ra một lỗi nghiêm trọng: Các lượt chạy trước lưu cache lời giải vào thư mục dùng chung EliteArchive. Khi lượt chạy sau thực thi, nó vô tình nạp lại lời giải tối ưu của lượt trước (warm-start). Điều này tạo ra kết quả đẹp ảo (ví dụ: số lượng xe giảm ngoạn mục), nhưng khi đem sang máy khác chạy từ đầu thì hoàn toàn không lặp lại được!
- Quyết định kỷ luật:** Bắt buộc xóa sạch cache, thiết lập nguyên tắc **Independent Cold-Starts** (luôn khởi động nguội độc lập từ thư mục trống). Mọi kết quả công bố phải tái lập được 100% từ con số 0.

2.2. Giai đoạn 1: Ý Tưởng Logprob (Prompt X & Y) — Kết Quả Âm Đắt Giá

- Ý tưởng ban đầu:** Khi LLM sinh ra từng chữ (token), nó có một chỉ số xác suất gọi là **logprob** (độ tự tin của mô hình). Người ta thường nghĩ: “Nếu mô hình tự tin (logprob cao) thì nó đúng, nếu tự tin thấp thì cho leo thang!”
- Kết quả thực nghiệm (5-fold Cross Validation × 20 lần lặp):** Thất bại hoàn toàn! Chỉ số $\Delta\text{AUROC} < 0.05$ và khoảng tin cậy chứa số 0.
- Tại sao thất bại?** Mô hình ngôn ngữ rất hay mắc chứng “**tự tin thái quá**” (**overconfidence**). Nó có thể tự tin 99% khi viết ra một dòng lập luận hoàn toàn sai logic! Do đó, logprob không thể dùng làm người gác cổng.

2.3. Giai đoạn 2: Ý Tưởng Đồng Thuận Dị Thể (Prompt AJ) — Cú Lừa “Verifier Miễn Phí”

Nhiều nghiên cứu gợi ý: Thay vì viết bộ kiểm tra riêng cho từng bài, hãy cho 2 hoặc 3 phương pháp khác nhau cùng giải (ví dụ PAL và CoT). Nếu chúng ra cùng một đáp án (đồng thuận), chắc chắn đáp án đó đúng! Ý tưởng này nghe rất quyến rũ vì không tốn công lập trình verifier. Nhưng khi kiểm nghiệm thực tế trên 100 câu:

Ví Dụ Thực Tế: Cú Lừa Đồng Thuận Ở Phép Toán Số Học (Arithmetic)

- Trong bài toán số học: Phương pháp **PAL** (dùng Python) đạt độ chính xác **100%** (40/40 câu đúng tuyệt đối), tốn chỉ **229 tokens**.
- Phương pháp **CoT** (nhẩm bằng lời văn) chỉ đúng **75%** (sai 10 câu do tính nhầm ẩu).
- Khi áp dụng luật đồng thuận: Ở 10 câu mà CoT nhầm sai, PAL và CoT **không đồng thuận** → Hệ thống bắt buộc phải kích hoạt phương pháp đắt tiền ToT để giải lại!
- Hậu quả bi hài:** Độ chính xác bị kéo tụt từ **100% xuống còn 75%**, trong khi lượng token bị đốt tăng thêm **+260%**! Một chuyên gia toán học giải đúng 100% lại bị ép làm lại bài chỉ vì một anh bạn tính nhầm sai bên cạnh không đồng ý!

Ví Dụ Thực Tế: Ảo Giác Tương Quan Ở Bài Toán Thứ Tự (Correlated Hallucination)

Ở bài toán sắp xếp logic thứ tự (câu order_0010):

- Đề bài hỏi: “Ai về đích ở vị trí thứ 6?” (Đáp án đúng là **Grace**).
 - Cả 3 phương pháp độc lập (PAL, CoT, ToT) cùng bị lừa bởi một cụm từ tiếng Anh lắt léo (“*ahead of*”) → Cả 3 cùng trả lời sai là **Alice**!
 - Hệ thống đồng thuận thấy 3/3 phiếu trùng nhau nên hờ hững chấp nhận đáp án sai!
- ⇒ **Kết luận:** Đồng thuận giữa các mô hình không thể thay thế một Verifier kiểm tra chứng chỉ độc lập!

2.4. Giai đoạn 3: Chạy Thực Tế Trên GPU (Pilot N=2 & N=8) & Phát Hiện Hiện Tượng HARM

Khi đưa mô hình thực tế Qwen3-8B-4bit lên chip Apple Silicon với cơ chế ghi đĩa an toàn (fsync journal), nhóm nghiên cứu đã quan sát được hiện tượng khoa học quan trọng nhất của toàn bộ đề tài:

Nghịch Lý HARM: Khi Verifier Đúng Nhưng Hệ Thống Lại Thua (False Reject → HARM)

Hãy xem điều gì đã xảy ra tại câu order_0001:

- Bước 1 (PAL sinh mã):** Mô hình Qwen3 viết một script Python. Trong đề bài có câu: “Erin về đích trước Frank đúng 2 vị trí”. Nhưng mô hình dịch nhầm thành: `abs(pos(Erin) - pos(Frank)) != 3`.
- Bước 2 (Verifier phát hiện):** Bộ kiểm tra độc lập (Verifier) chạy và kiểm tra chứng chỉ hoán vị. Nó phát hiện ngay: hoán vị này vi phạm dữ kiện số 3 → Gán nhãn **INVALID (Không hợp lệ)**. Verifier đã làm việc hoàn toàn chính xác!
- Bước 3 (Nghịch lý):** Mặc dù đoạn code của PAL bị lỗi logic, nhưng do may mắn ngẫu nhiên, khi in ra người về đích thứ 4, nó lại in đúng tên: **Frank** (đúng đáp án chân lý)!
- Bước 4 (Hậu quả thảm họa):** Vì Verifier từ chối PAL, hệ thống bắt buộc phải leo thang sang ToT fallback. Nhưng lúc này, bộ chọn của ToT lại gặp lỗi định dạng cú pháp (nó sinh ra Answer: 2 thay vì Best: 2) → Trả về rỗng → Câu hỏi bị chấm SAI!
- Tổng kết:** Verifier từ chối một lời giải nguy hiểm (nhưng tình cờ đoán mò đúng), đẩy sang fallback bị lỗi, khiến kết quả từ **ĐÚNG biến thành SAI**, đồng thời tốn thêm **+3,733 tokens** một cách oan uổng!

Trực Quan Hóa: Tại Sao Phát Hiện HARM Lại Đáng Giá Từng Xu?

Trong các bài báo nghiên cứu, người ta thường ca ngợi Verifier một chiều. Nhưng thí nghiệm của chúng ta chỉ ra một sự thật trần trụi: **Một Verifier thông minh kết hợp với một cơ chế Fallback yếu kém sẽ gây hại (HARM) cho toàn bộ hệ thống!** Đây là đóng góp học thuật cốt lõi giúp bài báo có chiều sâu phân biện thực thụ.

2.5. Giai đoạn 4: Shadow Replay (Thử Nghiệm Giả Định “What-If”)

Trước khi quyết định có nên tốn điện và thời gian chạy lại GPU hay không, nhóm thực hiện phân tích **Shadow Replay** trên 8 câu đã chạy:

- Nếu ta sửa bộ bóc tách Markdown (đọc được cả ****Answer: Grace****) → Phương pháp Verified tăng từ 4/8 lên **5/8**.
- Nếu ta thêm luật: “*Không bao giờ chọn nhánh bị cắt đứt do hết token, và nếu nhánh được chọn bị lỗi thì tự động chuyển sang nhánh hoàn chỉnh duy nhất*” → Phương pháp Verified tăng lên **6/8**!
- Điều này chứng minh: Không cần tăng ngân sách token hay đổi prompt mô hình vội vã; việc chuẩn hóa bộ hợp đồng **Contract v3 offline** sẽ cứu vãn được hệ thống.

2.6. Giai đoạn 5: Thách Thức Sống Còn Từ Bộ Giải Hình Thức (Symbolic Solver)

Điểm Nghẽn Học Thuật: Tại Sao Cần LLM Khi Python Thuần Giải Được Trong 3 Mili-Giây?

Khi nhóm chạy thử nghiệm hàm giải ràng buộc thuần túy (`solve_order_query`):

- Nó giải đúng tuyệt đối **31/31 câu development (100%)!**
- Thời gian chạy: **3.14 phần nghìn giây (millisecond)** trên CPU thông thường.
- Số token LLM tiêu tốn: **Đúng 0 token** (hoàn toàn miễn phí)!

Câu hỏi phản biện hóc búa của hội đồng khoa học: “Nếu một đoạn code Python 50 dòng chạy 3 mili-giây đã giải được 100% bài toán, tại sao các bạn lại phải tốn hàng ngàn token để gọi một mô hình LLM 8 tỷ tham số?”

Câu trả lời khoa học của đề tài:

1. **Giới hạn của Symbolic Solver:** Solver cứng chỉ giải được khi bài toán được đóng khung trong mẫu câu cứng nhắc (“*X finished ahead of Y*”). Nếu bài toán đưa vào ngôn ngữ tự nhiên đời thường, đầy ẩn ý và ngữ cảnh phong phú, solver cứng sẽ vỡ trận ngay lập tức ở khâu phân tích cú pháp.
2. **Vị trí đích thực của LLM + VGC:** LLM đóng vai trò là cây cầu chuyển hóa ngôn ngữ tự nhiên linh hoạt thành mã thực thi hình thức (Formal Code / Certificate). Còn Verifier đóng vai trò là tấm khiên bảo vệ tuyệt đối: Đảm bảo code do LLM sinh ra tuân thủ logic, loại bỏ hoàn toàn các lời giải nguy hiểm mà các hệ thống thông thường không phát hiện được.

3. Bảng Số Liệu Toàn Diện & Hướng Dẫn Đọc Dễ Hiểu

3.1. Bảng 1: Kiểm Định Cổng Đón Nhận Đồng Thuận (Agreement Gate)

Bảng này trả lời câu hỏi: *Tín hiệu đồng thuận có giúp dự đoán xem mô hình có làm đúng hay không?*

Họ bài	Tín hiệu kiểm tra	AUROC Gốc	AUROC Mới	Δ AUROC	95% Paired CI
arith	Đồng thuận CoT/ToT	Suy biến (PAL đúng 100% nên không có gì để phân loại)			
order	agree(PAL, CoT)	0.7175	0.7805	+0.0630	[−0.0629, +0.1928] (Trượt)
	agree(PAL, ToT)	0.7175	0.8370	+0.1195	[−0.0253, +0.2758] (Trượt)
	n_agree (PAL)	0.7175	0.8582	+0.1407	[+ 0.0221 , + 0.2775] (Đạt)
g24	Đồng thuận CoT/ToT	Suy biến (PAL sai 100% vì không sinh được biểu thức 24)			
pooled	n_agree (PAL)	0.9767	0.9899	+0.0132	[+0.0014, +0.0320] (Trượt vì $\Delta < 0.05$)

Table 1: Kiểm định AUROC (Diện tích dưới đường cong ROC) của tín hiệu đồng thuận.

Cách đọc bảng này cho người mới:

- **AUROC** là thang điểm từ 0.5 (đoán mò) đến 1.0 (tiên tri hoàn hảo).
- Muốn một tín hiệu được công nhận là có giá trị, quy tắc đặt trước (preregistration gate) yêu cầu 2 điều kiện: Δ AUROC phải tăng ít nhất **+0.05**, và cận dưới của khoảng tin cậy 95% phải **lớn hơn 0**.
- Ở bài order, chỉ có tín hiệu tổng hợp **n_agree** là vượt qua cổng thăm dò. Nhưng khi gộp chung (pooled), nó chỉ tăng vồn vẹn +0.0132 (quá yếu) → Trượt cổng toàn diện!

3.2. Bảng 2: So Sánh Hiệu Quả Các Chính Sách (Replay Policies)

Bảng này trả lời câu hỏi: *Nếu dùng các chính sách khác nhau trên cùng 100 câu hỏi cũ, ai thắng ai thua?*

Họ bài toán	Chính sách áp dụng	Độ chính xác	Tokens TB	Leo thang	Lời bình dễ hiểu
arith ($N = 40$)	PAL thuần túy	100.0%	229.2	0%	Nhanh nhất, rẻ nhất, đúng tuyệt đối
	Cổng thực thi W	100.0%	229.2	0%	Thông minh: không leo thang vô cớ
	Chính sách Đồng thuận	75.0%	824.0	25.0%	Kéo tụt độ chính xác, tốn gấp 3.6 lần tiền
order ($N = 31$)	PAL thuần túy	64.5%	474.3	0%	Nhanh nhưng sai 35% số câu
	Cổng thực thi W	74.2%	916.0	19.4%	Cứu được các câu crash code, rất tiết kiệm
	Chính sách Đồng thuận	71.0%	2,228.5	54.8%	Leo thang quá đà, tốn token khủng khiếp
pooled ($N = 100$)	Cổng thực thi W	80.0%	932.9	35.0%	Vô địch toàn diện về cả độ chính xác & chi phí
	Chính sách Đồng thuận	69.0%	1,577.7	56.0%	Vừa kém chính xác, vừa đắt đỏ

Table 2: Đối đầu thực nghiệm: Cổng thực thi W hoàn toàn áp đảo chính sách đồng thuận.

3.3. Bảng 3: Kết Quả Thử Nghiệm Mô Hình Thật Trên GPU (Pilot N=8)

Phương pháp	Đo thật v2	Shadow A	Shadow B	Tokens TB	Bản chất hiện tượng
PAL-answer (Chỉ lấy tên)	4 / 8	4 / 8	4 / 8	266	Code sinh đúng 1 tên
PAL-certificate (Chứng chỉ)	4 / 8	4 / 8	4 / 8	411	Code sinh đủ bảng thứ tự
ToT (3 nhánh + Selector)	3 / 8	4 / 8	5 / 8	5,226	Cực kỳ đắt (gấp 13 lần PAL)
Native-thinking (Tự suy nghĩ)	0 / 8	0 / 8	0 / 8	2,150	Sai 100% vì cạn trần 2,048 token
W-certificate (Cổng thực thi)	4 / 8	5 / 8	5 / 8	1,186	Chỉ leo thang 1 câu bị lỗi code
Verified-certificate (VGC)	4 / 8	5 / 8	6 / 8	3,627	Leo thang 5 câu bị sai logic
Symbolic Solver (Python thuần)	8 / 8	8 / 8	8 / 8	0	Giải hoàn hảo trong 1.4 mili-giây

Table 3: Đo đạc mô hình Qwen3-8B trên 8 câu development và tiềm năng cứu vãn của Contract v3.

4. Kế Hoạch Triển Khai Contract v3 Offline: Sửa Chữa Tận Gốc

Tại sao chúng ta không vội vã chạy lại GPU mà phải hoàn thiện Contract v3 offline trước? Bởi vì nếu không sửa các lỗi cú pháp ngớ ngẩn này, việc chạy thêm GPU chỉ làm lãng phí thời gian và tiền bạc mà không thu được giá trị khoa học nào mới.

Dưới đây là 3 quy tắc vàng của **Contract v3**:

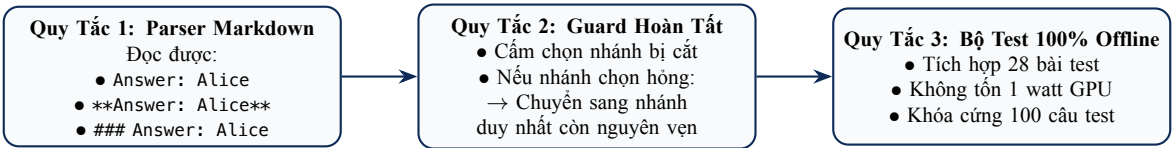


Figure 1: Sơ đồ 3 trụ cột cải tiến trong Contract v3.

1. Bóc tách định dạng Markdown linh hoạt nhưng nghiêm ngặt:

- Mô hình ngôn ngữ thời nay rất thích in đậm chữ: thay vì in Answer: Alice, nó hay in **Answer: Alice** hoặc **Answer:** Alice.
- Parser cũ bị mù trước các dấu sao này và coi như mô hình không trả lời.
- Contract v3 bóc tách sạch sẽ các định dạng Markdown này, nhưng kiên quyết từ chối nếu tên vận động viên không nằm trong danh sách của đề bài, hoặc nếu mô hình in ra nhiều tên mâu thuẫn.

2. Bộ chọn thông minh có nhận thức về độ hoàn tất (Completion-Aware Guard):

- Nếu một nhánh suy luận bị ngắt giữa chừng do chạm trần token (finish_reason: length) → Tuyệt đối cấm không được chọn nhánh này!
- Nếu bộ chọn ToT vô tình chỉ vào một nhánh bị ngắt, hệ thống sẽ kiểm tra: Trong 2 nhánh còn lại, có đúng duy nhất 1 nhánh đã hoàn tất trọn vẹn hay không?
- Nếu có đúng 1 nhánh hoàn tất → Tự động chuyển quyền sang nhánh đó. Nếu có 0 hoặc 2 nhánh → Kiên quyết từ chối (abstain) chứ không đoán mò.

3. Bảo đảm nguyên tắc khoa học tuyệt đối trước khi chạy chính thức:

- Toàn bộ 28 bài kiểm tra biên cú pháp phải chạy qua 100% trên môi trường offline.
- Tập dữ liệu kiểm chứng chính thức (100 câu test) tiếp tục được **niêm phong tuyệt đối (blinded & frozen)**, không được mở ra xem trước.

5. Tổng Kết 5 Bài Học Đắt Giá Cho Đề Tài NCKH

Nếu bạn phải trình bày đề tài này trước hội đồng bảo vệ, đây là 5 luận điểm đáng thép nhất mà bạn cần ghi nhớ:

- Giá trị của kết quả âm (The Value of Negative Results):** Đề tài không tô hồng số liệu. Chúng ta đã chứng minh bằng toán học và thực nghiệm rằng: (1) Logprob không dùng để định tuyến suy luận đa bước được; (2) Cơ chế đồng thuận không phải là “verifier miễn phí” mà là một cái bẫy làm giảm độ chính xác và đội chi phí lên gấp nhiều lần. Đây là những đóng góp khoa học có giá trị phản biện rất cao.
- Bản chất hai mặt của Verifier và Hiện tượng HARM:** Một Verifier kiểm tra chứng chỉ hình thức là vũ khí tối thượng để loại bỏ các lời giải nguy hiểm (Invalid Witness). Tuy nhiên, nếu hệ thống dự phòng (Fallback) không đủ vững chắc, Verifier có thể gây hại cho hệ thống khi nó bác bỏ một lời giải tình cờ đoán đúng.
- Cổng Thực Thi W là đối chứng cực mạnh (Execution Gate Baseline):** Chỉ bằng việc kiểm tra xem code Python có chạy lỗi hay không (try/except), cổng W đã đạt độ chính xác 80% với chi phí cực thấp (932 tokens), vượt xa các cơ chế đồng thuận phức tạp. Bất kỳ tăng thác VGC nào muốn khẳng định giá trị đều bắt buộc phải thắng được cổng W.
- Tính trung thực khoa học với Symbolic Solver:** Chúng ta công khai thừa nhận một hàm Python 3 mili-giây có thể giải 100% tập bài toán có mẫu cố định. Từ đó, định vị rõ ràng giá trị của LLM: LLM không phải để thay thế máy tính làm tính toán số học, mà để làm cầu nối hiểu ngôn ngữ tự nhiên phức tạp trước khi bàn giao cho bộ xác thực hình thức.
- Phương pháp luận thực nghiệm không tì vết:** Mọi kết quả đều được bảo vệ bởi nguyên tắc khởi động nguội (Independent Cold-Starts), nhật ký fsync chống gián đoạn, mã nguồn mở tái lập được 100%, và tập dữ liệu kiểm nghiệm 100 câu độc lập vẫn được khóa kín cho đến ngày nghiệm thu cuối cùng.

KẾT THÚC TÀI LIỆU HƯỚNG DẪN TỔNG QUAN

Biên soạn bởi Nhóm Nghiên Cứu Multi-Step Structured Reasoning · 08/10/2026